



CS1004 – Object Oriented Programming

Lecture # 11
Monday, March 21, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

This week

- Static members and functions
- Constant members and this pointer

Static data member

- ➡ A variable that is part of a class, yet is not part of an object of that class, is called static data member

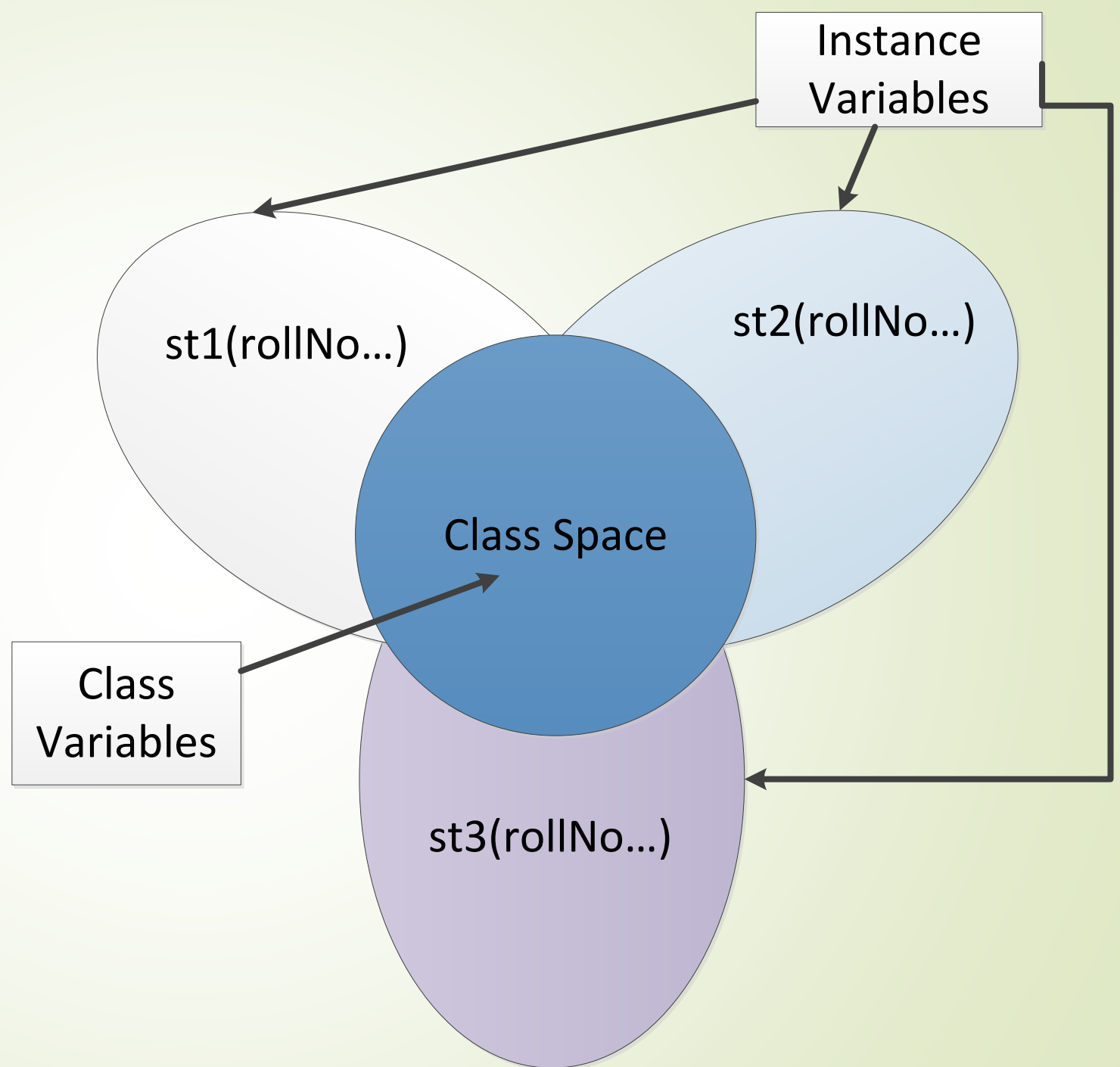
Declaring **static** data member syntax

- ➡ **static** keyword is used to make a data member static

```
class ClassName
{
    ...
    static dataType variableName;
    ...
};
```

Class vs instance variable

➡ Student st1,
st2, st3;



Static data members

- They are shared by all instances of the class
- They do not belong to any particular instance, of a class

Defining **static** data members

- ➡ Static data member is declared inside the class
- ➡ But they are defined outside the class

Defining **static** data members

```
class ClassName
{
    ...
    static dataType variableName;
    ...
};
```

```
dataType ClassName::variableName;
```


Example

```
class Student {  
    private:  
        static int noOfStudents;  
    public:  
        ...  
};  
int Student::noOfStudents = 0;  
/*private static member cannot be accessed  
outside the class except for initialization*/
```

Initializing **static** data member

- ➡ If static data members are not explicitly initialized at the time of definition then they are initialized to 0

Example

```
int Student::noOfStudents;
```

is equivalent to

```
int Student::noOfStudents = 0;
```

Accessing **static** data member

- To access a static data member there are two ways
 - Access like a normal data member
 - Access using a scope resolution operator "::"

Example

```
class Student {  
public:  
    static int noOfStudents;  
};  
int Student::noOfStudents;  
int main() {  
    Student aStudent;  
    aStudent.noOfStudents = 1;  
    Student::noOfStudents = 1;  
}
```

Life of data **static** members

- They are created even when there is no object of a class
- They remain in memory even when all objects of a class are destroyed

Example

```
class Student {  
public:  
    static int noOfStudents;  
};  
int Student::noOfStudents;  
int main() {  
    {  
        Student aStudent;  
        aStudent.noOfStudents = 1;  
    }  
    Student::noOfStudents = 1;  
}
```

Uses of **static** variable

- ➡ They can be used to store information that is required by all objects, like global variables

Example

- ➡ Modify the class Student such that one can know the number of student created in a system

Example

```
class Student {  
public:  
    static int noOfStudents;  
    Student();  
    ~Student();  
};  
int Student::noOfStudents = 0;  
Student::Student() {  
    noOfStudents++;  
}  
Student::~~Student() {  
    noOfStudents--;  
}
```

```
int Student::noOfStudents = 0;  
int main() {  
    cout << Student::noOfStudents  
        << endl;  
    Student studentA;  
    cout << Student::noOfStudents  
        << endl;  
    Student studentB;  
    cout << Student::noOfStudents  
        << endl;  
}
```

Output:

0
1
2

Static member function

- ➡ Static member function can access only static data members or functions

Example

```
#include<iostream>
using namespace std;
class Student
{
    static int noOfStudents;
    int rollNo;
public:

    Student();
    int getRollNo();
    void setRollNo(int x);
    static int getStudentCount();
};
//noOfStudents set to 15
int Student::noOfStudents = 15;
Student::Student(){
    rollNo = 0;
}

int Student::getRollNo(){
    return rollNo;
}
```

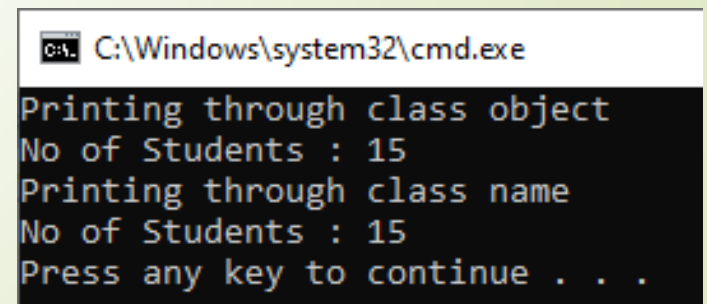
CS1004 - Objected Oriented Programming

```
void Student::setRollNo(int x){
    rollNo = x;
}
int Student::getStudentCount(){
    return noOfStudents;
}

int main(){
    Student st1;
    st1.setRollNo(20);
    cout << "Printing through class object" << endl;
    cout << "No of Students : "
        << st1.getStudentCount()<<endl;

    cout << "Printing through class name" << endl;
    cout << "No of Students : "
        << Student::getStudentCount()<<endl;

    return 0;
}
```



```
C:\Windows\system32\cmd.exe
Printing through class object
No of Students : 15
Printing through class name
No of Students : 15
Press any key to continue . . .
```

Array Of Objects

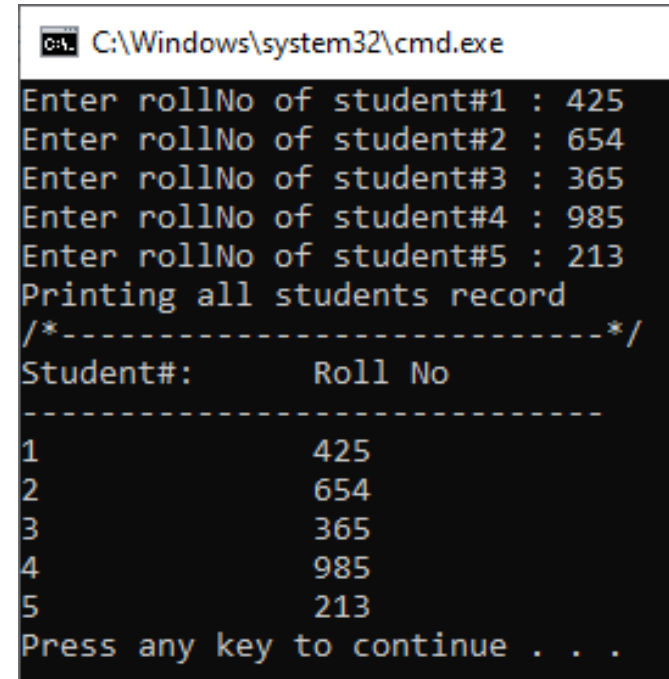
- An object of class represents a single record in memory, if we want more than one record of class type, we have to create an array of objects
- As we know, an array is a collection of similar type, therefore an array can be a collection of class type

Example

```

int main()
{
    Student students[5];
    int temp;
    for (int i = 0; i < 5; ++i)
    {
        cout << "Enter rollNo of student#" << i + 1 << " : ";
        cin >> temp;
        students[i].setRollNo(temp);
    }
    cout << std::left;
    cout << "Printing all students record" << endl
        << "/*-----*/" << endl
        << setw(15)<<"Student#:" <<setw(15)<< "Roll No" << endl
        << "-----" << endl;
    for (int i = 0; i < 5; ++i)
        cout << setw(15) << i + 1
            << setw(15)<<students[i].getRollNo() << endl;
    return 0;
}

```



```

C:\Windows\system32\cmd.exe

Enter rollNo of student#1 : 425
Enter rollNo of student#2 : 654
Enter rollNo of student#3 : 365
Enter rollNo of student#4 : 985
Enter rollNo of student#5 : 213
Printing all students record
/*-----*/
Student#:      Roll No
-----
1             425
2             654
3             365
4             985
5             213
Press any key to continue . . .

```

Pointer to object

- ➡ A variable that holds an address value is called a pointer variable or simply pointer
- ➡ Pointer to objects are similar as pointer to builtin types

Example

```
int main()
{
    Student st1;
    Student *stPtr;
    stPtr = &st1;
    st1.setRollNo(99);
    (*stPtr).setRollNo(99);
    stPtr->setRollNo(99);
}
```


Allocation with **new** operator

- Sometimes we do not know, at the time that we write the program , how many objects we want to create
- When this is the case, we can use new to create objects while the program is running
- “**new**” returns a pointer to unnamed objects

Example

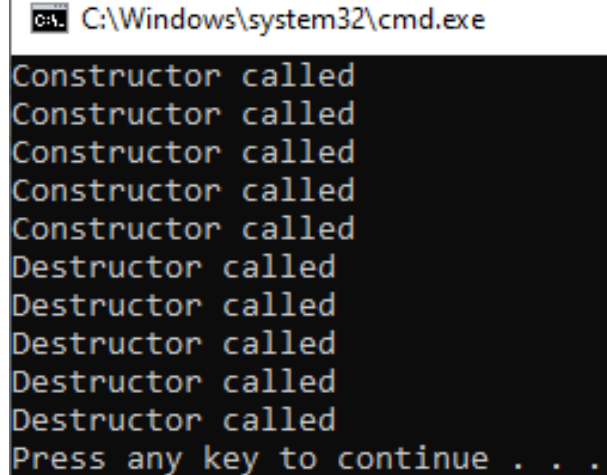
```
int main() {  
    Student *ptr;  
    ptr = new Student;  
    ptr->setRollNo(1050);  
    return 0;  
}
```

deallocating with `delete` operator

```
int main() {  
    Student *stPtr;  
    stPtr = new Student;  
    stPtr->setRollNo(1050);  
    delete stPtr;  
    return 0;  
}
```

Dynamic array of objects

```
class Student {  
    int rollNo;  
public:  
    Student() { cout << "Constructor called" << endl; }  
    ~Student() { cout << "Destructor called" << endl; }  
};  
int main() {  
    Student *stPtr;  
    stPtr = new Student[5];  
    delete[] stPtr;  
    return 0;  
}
```



```
C:\Windows\system32\cmd.exe  
Constructor called  
Constructor called  
Constructor called  
Constructor called  
Constructor called  
Destructor called  
Destructor called  
Destructor called  
Destructor called  
Destructor called  
Press any key to continue . . .
```

Class Task

- Design a class date through which user must be able to perform following operations
 - Get and set current day, month and year
 - Increment by x number of days, months and year
 - Set default date

Attributes

- Attributes that can be seen in this problem statement are
 - Day
 - Month
 - Year
 - Default date
- The default date is a feature shared by all objects
 - This attribute must be declared a static member

Attribute in DATE.H

```
class Date
{
    int day;
    int month;
    int year;
    static Date defaultDate;
};
```

Interfaces

- `getDay`
- `getMonth`
- `getYear`
- `setDay`
- `setMonth`
- `setYear`

- `addDay`
- `addMonth`
- `addYear`
- `setDefaultDate`

Interfaces

```
class Date {  
    ...  
public:  
    void setDay(int aDay);  
    int getDay() const;  
    void addDay(int x);  
    ...  
    ...  
};
```

Interfaces

- ➡ As the default date is a static member the interface `setDefaultDate` should also be declared static

```
class Date {  
    ...  
public:  
    static void setDefaultDate(  
        int aDay, int aMonth, int aYear);  
    ...  
};
```

Implementation of Date class

- ➡ The static member variables must be initialized

```
Date Date::defaultDate(01, 01, 2019);
```

Constructor

```
Date::Date(int aDay, int aMonth, int aYear)
{
    if (aDay == 0) {
        this->day = defaultDate.day;
    }
    else {
        setDay(aDay);
    }
    //similarly for other members
}
```

Destructor

```
Date::~~Date()
```

```
{
```

```
}
```

Getter and Setter

```
void Date::setMonth(int a) {  
    if (a > 0 && a <= 12) {  
        month = a;  
    }  
}
```

Questions

